SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Ashwanth SP/192126002 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA-1709

DONE BY: ASHWANTH SP (192525014)

CONSTRAINT-BASED PROBLEM SOLVING

QUESTION 1: DOCTOR SHIFT ASSIGNMENT USING BACKTRACKING SEARCH

The task is to assign doctors D1, D2 and D3 to the Morning, Afternoon and Night shifts. Each doctor must receive exactly one shift, and each shift must contain exactly one doctor. D1 cannot work the Night shift, D2 must work before D3, and D3 cannot work the Morning shift. Backtracking Search assigns a shift to one doctor at a time, checks the constraints immediately, and returns to an earlier decision if a constraint is violated.

Step 1: Assign D1 to Morning. This is valid because D1 is not assigned to Night. Step 2: Assign D2 to Afternoon. This shift is still available, and D2 can work before D3. Step 3: Assign D3 to Night. D3 is not assigned to Morning, and D2's Afternoon shift occurs before D3's Night shift. All shifts are distinct and every doctor has one shift, so the assignment satisfies every constraint and no backtracking is required.

Step	Decision	Constraint check
1	D1 → Morning	Valid: D1 is not on Night.
2	D2 → Afternoon	Valid: shift is unused; D2 is before D3.
3	D3 → Night	Valid: D3 is not on Morning and follows D2.
Result	Morning: D1; Afternoon: D2; Night: D3	All constraints are satisfied.

Final answer: The valid schedule is Morning—D1, Afternoon—D2, and Night—D3. Each doctor has exactly one shift, no shift is shared, D1 avoids Night, D3 avoids Morning, and D2 works before D3.

QUESTION 2: 5 × 5 GRID NAVIGATION USING BFS

Breadth-First Search (BFS) explores all reachable cells at distance 0, then distance 1, distance 2, and so on. It uses a first-in, first-out (FIFO) queue. Because every movement costs 1, the first time BFS reaches the goal it has found a path with the minimum number of moves. The robot can move Up, Down, Left or Right, cannot enter blocked cells, and records each cell's parent so the route can be

reconstructed. Manhattan distance is h(n) = |row(n) − row(G)| + |column(n) − column(G)|. It estimates the remaining distance; standard BFS does not use this heuristic to order its queue.

Grid used for the worked solution (S = start, G = goal, # = obstacle, . = free cell):

S	.	#	.	.
.	.	#	.	#
#	.	.	.	.
.	#	#	#	.
.	.	.	.	G

Coordinates use (row, column), starting at (0,0). S = (0,0), G = (4,4). Neighbours are examined in this fixed order: Right, Down, Left, Up. A cell is marked visited when it is first enqueued, preventing duplicate queue entries.

Step-by-step node expansion and queue updates

Expansion 1: remove (0, 0) (cost 0). Newly enqueued: (0, 1) (cost 1), (1, 0) (cost 1). Queue after expansion: (0, 1), (1, 0).

Expansion 2: remove (0, 1) (cost 1). Newly enqueued: (1, 1) (cost 2). Queue after expansion: (1, 0), (1, 1).

Expansion 3: remove (1, 0) (cost 1). Newly enqueued: No new nodes. Queue after expansion: (1, 1).

Expansion 4: remove (1, 1) (cost 2). Newly enqueued: (2, 1) (cost 3). Queue after expansion: (2, 1).

Expansion 5: remove (2, 1) (cost 3). Newly enqueued: (2, 2) (cost 4). Queue after expansion: (2, 2).

Expansion 6: remove (2, 2) (cost 4). Newly enqueued: (2, 3) (cost 5). Queue after expansion: (2, 3).

Expansion 7: remove (2, 3) (cost 5). Newly enqueued: (2, 4) (cost 6), (1, 3) (cost 6). Queue after expansion: (2, 4), (1, 3).

Expansion 8: remove (2, 4) (cost 6). Newly enqueued: (3, 4) (cost 7). Queue after expansion: (1, 3), (3, 4).

Expansion 9: remove (1, 3) (cost 6). Newly enqueued: (0, 3) (cost 7). Queue after expansion: (3, 4), (0, 3).

Expansion 10: remove (3, 4) (cost 7). Newly enqueued: (4, 4) (cost 8). Queue after expansion: (0, 3), (4, 4).

Expansion 11: remove (0, 3) (cost 7). Newly enqueued: (0, 4) (cost 8). Queue after expansion: (4, 4), (0, 4).

Expansion 12: remove (4, 4) (cost 8). Newly enqueued: No new nodes. Queue after expansion: (0, 4).

Optimal path: (0, 0) → (0, 1) → (1, 1) → (2, 1) → (2, 2) → (2, 3) → (2, 4) → (3, 4) → (4, 4).

Total path cost: 8 moves × 1 per move = 8. The path is optimal because BFS processes all nodes in increasing order of depth and reaches the goal at the smallest possible number of moves.

BFS algorithm: Put S into the queue and mark it visited. While the queue is not empty, remove the front cell. If it is G, reconstruct the route by following parent links and stop. Otherwise, inspect its four neighbours. Enqueue each neighbour only if it lies inside the grid, is not an obstacle, and has not been visited. Record its parent and its cost as the current cost plus 1. If the queue becomes empty before G is reached, no path exists.

QUESTION 3: AUTONOMOUS RESCUE ROBOT IN A PARTIALLY OBSERVABLE BUILDING

(a) Analysis of constraints

The robot must travel from S to the survivor at G in a building represented as a grid. It may move only Up, Down, Left or Right, and blocked cells must not be entered. Its sensors reveal only adjacent cells, so the complete layout is not initially known. The robot must build and update its map as it explores. Every ordinary movement costs 1, while entering a risky zone adds 2, making that movement cost 3. Therefore, the robot must consider both distance and safety: the route with the fewest steps is not necessarily the route with the lowest total cost. New observations may reveal obstacles or risks, requiring the robot to revise its plan.

(b) Solution using Uniform Cost Search with online search

Uniform Cost Search (UCS) is appropriate because actions have different costs. The robot starts by inserting S into a priority queue with cumulative cost 0. It senses adjacent cells and records whether they are open, blocked or risky. The algorithm repeatedly removes the state with the smallest known cumulative cost. For each legal neighbour, it adds 1 for a normal move or 3 for entering a risky zone. If this produces a cheaper route to a cell than the route recorded previously, the robot updates that cell's cost and parent. When G is reached, the parent links identify the least-cost route through the known map.

Because the robot has only local sensing, it should use an online cycle: sense adjacent cells, update the internal map, plan using the known information, take one safe action, and sense again. If a new obstacle or risky zone is discovered, it updates the map and replans rather than following a route that is no longer valid. UCS is optimal for a known map with non-negative costs. In this partially observable scenario, the plan is optimal with respect to the information currently available, and it may be revised as new observations arrive.

(c) Application of AI concepts

Intelligent agent: The rescue robot is an intelligent agent because it perceives its surroundings through sensors and acts through movement to achieve the goal of reaching the survivor. Rationality: A rational agent selects actions expected to achieve the goal with the lowest cost while respecting safety constraints, based on the information available. It does not automatically choose the route with the fewest steps; it compares cumulative costs and avoids risky zones when a cheaper safe route exists. Environment type: The building is partially observable because only adjacent cells can be sensed; sequential because each move affects later decisions; discrete because the robot moves between grid cells; and initially unknown because the robot learns the map online. From the robot's

perspective, the environment changes as its knowledge is updated. The scenario does not specify random action outcomes, so stochastic behaviour need not be assumed.

(d) Justification

UCS is more suitable than BFS for this rescue task because the objective is to minimize total cost rather than simply the number of moves. A normal cell costs 1 to enter, while a risky cell costs 3. BFS treats all moves as equal and can therefore select a short route that is more expensive or less safe. UCS expands the frontier state with the lowest cumulative cost and finds a minimum-cost route when the map and non-negative costs are known. Combining UCS with online sensing and replanning handles partial observability by incorporating newly discovered obstacles and risk zones. This approach balances cost-aware navigation with the need to update decisions as the robot learns more about the building.

CONCLUSION

Backtracking Search finds a valid doctor schedule by checking constraints after each assignment. BFS finds the minimum-move path in a grid when every movement has equal cost. For the rescue robot, Uniform Cost Search combined with local sensing and online replanning is appropriate because risky zones have higher costs and the complete environment is not initially known.